72.9
Pass

# Satyabart Giri

*PDF generated at: 24 Feb 2026 06:34:11 UTC*

View this report on HackerRank

## Score

72.9% • 127.5 / 175
scored in eClerx - Java Coding Test - 3+ Years New in 24 min 48 sec on 24 Feb 2026 11:37:25 IST

## Candidate Information

| | |
|---|---|
| Email | gsatyabart@gmail.com |
| Test | eClerx - Java Coding Test - 3+ Years New |
| Candidate Packet | View |
| Taken on | 24 Feb 2026 11:37:25 IST |
| Time taken | 24 min 48 sec/ 90 min |
| Work Experience | 3 years |
| Invited by | Vipul |
| Invited on | 23 Feb 2026 21:14:26 IST |

## Skill Distribution

| No. | Skill | Score |
|---|---|---|
| 1 | Spring Boot Advanced | 66% |

| | | |
|---|---|---|
| 2 | Spring Boot<br>Intermediate | 75% |
| 3 | Problem Solving<br>Intermediate | 100% |
| 4 | Java<br>Intermediate | 46% |

## Tags Distribution

| | | | |
|---|---|---|---|
| Spring Boot | 75% | Hard | 66% |
| TDD | 100% | Medium | 73% |
| Spring | 50% | Pointers | 100% |
| Greedy | 46% | Sorting | 46% |
| Java | 46% | | |

## Questions

Spring Boot (Advanced) • 17.5 / 25

| No. | Question | Skill | Score | Code Quality | Time Taken | Status |
|---|---|---|---|---|---|---|
| 1 | Specification<br>Multiple Choice | Spring Boot (Advanced) | 5/5 | - | 31 sec | ✓ |

| | | | | | | |
|---|---|---|---|---|---|---|
| 2 | Determine the Output<br>Multiple Choice | Spring Boot (Advanced) | 0/5 | - | 1 min 46 sec | ✗ |
| 3 | Output<br>Multiple Choice | Spring Boot (Advanced) | 5/5 | - | 51 sec | ✓ |
| 4 | Spring Scopes<br>Multiple Choice | Spring Boot (Intermediate) | 2.5/5 | - | 24 sec | ✓ |
| 5 | CSRF<br>Multiple Choice | Spring Boot (Intermediate) | 5/5 | - | 21 sec | ✓ |

Problem Solving (Intermediate) • 75 / 75

| No. | Question | Skill | Score | Code Quality | Time Taken | Status |
|---|---|---|---|---|---|---|
| 6 | Quiz Competition<br>Coding | Problem Solving (Intermediate) | 75/75 | - | 10 min 59 sec | ✓ |

Java (Intermediate) • 35 / 75

| No. | Question | Skill | Score | Code Quality | Time Taken | Status |
|---|---|---|---|---|---|---|
| 7 | Java: DataVille's Optimal Storage<br>Coding | Java (Intermediate) | 35/75 | - | 9 min 32 sec | ✓ |

## Candidate Feedback

Good Exprience

## 1. Specification

Correct

Multiple Choice | Spring Boot | Hard

### Question description

Which of the following options can be used for dynamic jpa queries with spring-data-jpa repositories?

### Candidate's Solution

**Options:** (Expected answer indicated with a tick)

- ○ PreparedStatement
- ● Specification ✓
- ○ Criteria
- ○ Query

No comments.

## 2. Determine the Output

Incorrect

Multiple Choice | Spring Boot | Hard

## Question description

SpringQuestionApplication.java

```
package com.hackerrank.spring;
import org.springframework.boot.SpringApplication;
import org.springframework.boot.autoconfigure.SpringBootApplication;
@SpringBootApplication
public class SpringQuestionsApplication {
   public static void main(String[] args) {
      SpringApplication.run(SpringQuestionsApplication.class, args);
   }
}
```

Lie.java

```
package com.hackerrank;
import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;
@Component
public class Lie implements CommandLineRunner {
   @Override
   public void run(String... args) throws Exception {
      System.out.println("Truth is behind the lies");
      System.out.println("This is working perfectly");
      throw new Exception("Does this work?");
   }
}
```

Truth.java

```
package com.hackerrank.spring.reality;

import org.springframework.boot.CommandLineRunner;
import org.springframework.stereotype.Component;

@Component
public class Truth implements CommandLineRunner {
   @Override
   public void run(String... args) throws Exception {
      System.out.println("Truth is in your eyes");
```

```
    }
}
```

What will be printed on the console when the "SpringBootApplication" class is executed?

**Interviewer guidelines**

## Correct Answer:

**Option [A] – "Truth is in your eyes"**

**Explanation:** Spring Boot scans only the package of the `@SpringBootApplication` class and its subpackages. `Truth` is under a subpackage and runs. `Lie` is in a parent package, so it is **not scanned** and its runner does not execute.

---

## Incorrect Answers:

**Option [B] – Multiple prints + exceptions**

Explanation: `Lie` never runs, so its prints and exception do not occur.

**Option [C] – Prints from Lie only**

Explanation: `Truth` is discovered and executed; `Lie` is not.

**Option [D] – Prints from Lie, then Truth**

Explanation: `Lie` is outside the scan path and never executed.

**Candidate's Solution**

**Options:** (Expected answer indicated with a tick)

- Truth is in your eyes ✓
- Truth is in your eyes Truth is behind the lies This is working perfectly And exception occurs and application stops
- Truth is behind the lies This is working perfectly And exception occurs and application stops

Truth is behind the lies This is working perfectly And exception occurs containing “Does this work?” Truth is in your eyes

No comments.

## 3. Output

Correct

Multiple Choice | Hard | Spring Boot | TDD

### Question description

What will happen to the output file?

```
@SpringBootTest
public class LogTest
{
   File log;
   FileWriter logWriter;

   @Autowired
   SomeClassToTest someClassToTest;

   @Before
   public void init()
   {
     log = new File("log.txt");
     logWriter = new FileWriter(log);
   }

   @After
   public void teardown()
   {
      log.delete();
   }

   @Test
   public void test1()
```

```
    {
        logWriter.write("Entered Test 1");
        assertTrue(someClassToTest.someMethod());
    }

    @Test
    public void test2()
    {
        logWriter.write("Entered Test 2");
        assertTrue(someClassToTest.otherMethod());
    }
}
```

### Candidate's Solution

**Options:** (Expected answer indicated with a tick)

- ○ The output file is not created.
- ○ The output file is created but not deleted.
- ● **The output file is created and eventually deleted.** ✓
- ○ The create and delete blocks will not be executed.

No comments.

## 4. Spring Scopes

Partially correct

Multiple Choice | Medium | Spring

### Question description

Which of the following statements is/are true about @Scope("request") and @Scope("prototype") annotations?

### Candidate's Solution

**Options:** (Expected answer indicated with a tick)

- [ ] Both are used to define the scope of spring beans.
- [x] **@Scope("request") creates a bean instance for a single HTTP request.**
- [ ] @RequestScope can be used instead of @Scope("request").
- [x] **@Scope("prototype") will return a different instance every time it is requested from the container.**

**No comments.**

---

## 5. CSRF

Correct

Multiple Choice | Spring Boot | Medium

### Question description

Which Java Spring-based code segment correctly configures CSRF?

### Interviewer guidelines

CSRF Protection makes sure that server is not bombarded with unwanted calls. Spring Security enables some features by default like CSRF and Form based authentication.

### Candidate's Solution

**Options:** (Expected answer indicated with a tick)

- ○ private void manage(HttpSecurity http) {   http.csrf().allowed() }  
- ○ private void manage(HttpSecurity http) {   http.csrf() }  
- ○ private void manage(HttpSecurity http) {   http.csrf().enable() }  
- ● ** None of the above. CSRF is configured by default when Spring Security Dependency is added. ** ✓

No comments.

---

## 6. Quiz Competition

✓ Correct

Coding | Medium | Pointers

### Question description

A class has students with various talents, each represented by an integer from 1 to *talentsCount*. You need to form teams for a quiz competition, where each team must have at least one member with each talent.

Teams must be formed from consecutive students in the array. For each possible starting position, determine the minimum number of students needed to form a valid team. If it is not possible to form a team with all talents from a particular starting position, return -1 for that position.

### Example

*talentsCount* = 3

*talent* = [1, 2, 3, 2, 1]

- Starting at position 1: [1, 2, 3] includes all talents, minimum size = 3
- Starting at position 2: [2, 3, 2, 1] is the smallest subarray with all talents, minimum size = 4
- Starting at position 3: [3, 2, 1] includes all talents, minimum size = 3
- Starting at positions 4 and 5: Cannot form a team with all talents, return -1

The result is [3, 4, 3, -1, -1].

### Function Description

Complete the function *teamSize* in the editor with the following parameter(s):

*int talent[n]*: the students' talents

*int talentsCount* : the number of talents represented numbered from 1 to *talentsCount*

### Returns

*int[n]:* at each index, the minimum size subarray required, or -1 if an appropriate subarray does not exist

### Constraints

- $1 \le n, talentsCount \le 10^5$
- $1 \le talent[i] \le talentsCount$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *n*, the number of elements in *talent*.

Each line *i* of the *n* subsequent lines (where $0 \le i < n$) contains an integer, *talent[i]*.

The next line contains the integer *talentCount*.

▼ SAMPLE CASE 0

### Sample Input For Custom Testing

```
STDIN   Function
-----   --------
8       talent[] size n = 8
1       talent = [1, 1, 2, 2, 3, 1, 3, 2]
1
2
2
3
1
3
2
3       talentsCount = 3
```

### Sample Output

```
5
4
4
3
4
3
-1
-1
```

### Explanation

The shortest subarrays for each position:

Beginning at the first position: [1, 1, 2, 2, 3] (length 5)

Second position: [1, 2, 2, 3] (length 4)

Third position:  [2, 2, 3, 1] (length 4)

Fourth position: [2, 3, 1] (length 3)

Fifth position:  [3, 1, 3, 2] (length 4)

Sixth position: [1, 3, 2] (length 3)

No further subarrays will have 3 elements.

▼ SAMPLE CASE 1

### Sample Input For Custom Testing

```
STDIN   Function
-----   --------
5       talent[] size n = 5
1       talent = [1, 1, 1, 1, 1]
1
1
1
1
1       talentsCount = 1
```

### Sample Output

```
1
1
1
1
1
```

### Explanation

Since there is only 1 type of talent, any 1 person can form a team.

▼ SAMPLE CASE 2

### Sample Input For Custom Testing

```
STDIN   Function
-----   --------
9       talent[] size n = 9
7       talent = [7, 5, 3, 4, 6, 1, 7, 2, 4]
5
3
4
6
1
7
2
4
7       talentsCount  7
```

### Sample Output

```
8
7
-1
-1
```

```
-1
-1
-1
-1
-1
```

**Explanation**

There have to be at least *7* elements in any subarray to have one from each talent. The shortest subarrays at each position are:

Beginning at the first position: [7, 5, 3, 4, 6, 1, 7, 2] (length 8)

Second position: [5, 3, 4, 6, 1, 7, 2] (length 7)

No further subarrays will have 7 elements.

## Interviewer guidelines

▼ SOLUTION

This can be solved using two-pointer technique. For a particular left index (L), we move the right index (R) forward until we have k distinct elements, then once we get it, we can move L forward. Each element will be traversed at most two times.

Time Complexity: O(n) or O(nLogn) (with set/map).

C++ Solution:

```
int f[100005];

vector<int> teamSize(vector<int> talent, int talentsCount) {
    int k = talentsCount;
    int n = talent.size();
    assert(n>=1&&n<=100000);
    assert(k>=1&&k<=n);
    for(int i=0;i<n;i++)
    {
        assert(talent[i]<=k&&talent[i]>=1);
    }
    int cnt = 0;
    int j = 0;
    vector<int> ans(n);
```

```
    for(int i=0;i<n;i++)
    {
        int x = talent[i];
        f[x]++;
        if(f[x]==1)
            cnt++;
        while(cnt==k&&j<=i)
        {
            ans[j] = i - j + 1;
            int y = talent[j];
            f[y]--;
            if(f[y]==0)
                cnt--;
            j++;
        }
    }
    while(j<n)
        ans[j++] = -1;
    return ans;
}
```

**Candidate's Solution**

Language used: **Java 8**

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;



class Result {

    /*
     * Complete the 'teamSize' function below.
     *
```

```
     * The function is expected to return an INTEGER_ARRAY.
     * The function accepts following parameters:
     *  1. INTEGER_ARRAY talent
     *  2. INTEGER talentsCount
     */

    public static List<Integer> teamSize(List<Integer> talent, int talentsCount) {
    // Write your code here
    int n =talent.size();
    List<Integer>result=new ArrayList<>();

    Map<Integer,Integer>freq=new HashMap<>();
    int left =0,right=0;
    int uniqueCount=0;

    while(left<n){
        while (right<n&&uniqueCount<talentsCount){
            int val=talent.get(right);
            freq.put(val,freq.getOrDefault(val, 0)+1);
            if(freq.get(val)==1){
                uniqueCount++;
            }
            right++;
        }
        if(uniqueCount==talentsCount){
            result.add(right-left);
        }else{
            result.add(-1);

        }
        int leftVal=talent.get(left);
        freq.put(leftVal,freq.get(leftVal)-1);
        if(freq.get(leftVal)==0){
            uniqueCount--;
        }
        left++;
    }
    return result;

    }

}

public class Solution {
    public static void main(String[] args) throws IOException {
```

```java
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int talentCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> talent = IntStream.range(0, talentCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int talentsCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> result = Result.teamSize(talent, talentsCount);

        bufferedWriter.write(
            result.stream()
                .map(Object::toString)
                .collect(joining("\n"))
            + "\n"
        );

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

| TESTCASE | DIFFICULTY | TYPE | STATUS | SCORE | TIME TAKEN | MEMORY USED |
|---|---|---|---|---|---|---|
| TestCase 0 | Easy | Sample | Success | 1 | 0.0979 sec | 30.5 KB |

| | | | | | | |
|---|---|---|---|---|---|---|
| TestCase 1 | Easy | Sample | Success | 1 | 0.0972 sec | 30 KB |
| TestCase 2 | Easy | Sample | Success | 1 | 0.1021 sec | 30.3 KB |
| TestCase 3 | Easy | Hidden | Success | 3 | 0.1765 sec | 39.4 KB |
| TestCase 4 | Easy | Hidden | Success | 3 | 0.1487 sec | 39.6 KB |
| TestCase 5 | Easy | Hidden | Success | 3 | 0.159 sec | 38.7 KB |
| TestCase 6 | Easy | Hidden | Success | 8 | 0.3821 sec | 71.4 KB |
| TestCase 7 | Easy | Hidden | Success | 8 | 0.3519 sec | 69.8 KB |
| TestCase 8 | Easy | Hidden | Success | 8 | 0.412 sec | 76.7 KB |
| TestCase 9 | Easy | Hidden | Success | 8 | 0.3731 sec | 71.2 KB |
| TestCase 10 | Easy | Hidden | Success | 8 | 0.4533 sec | 59.7 KB |
| TestCase 11 | Easy | Hidden | Success | 8 | 0.4673 sec | 71.7 KB |
| TestCase 12 | Easy | Hidden | Success | 8 | 0.4181 sec | 66.9 KB |
| TestCase 13 | Easy | Hidden | Success | 7 | 0.43 sec | 57.2 KB |

No comments.

## 7. Java: DataVille's Optimal Storage

Partially correct

Coding Greedy Sorting Medium Java

### Question description

In the digital city of DataVille, efficient data management is critical. The city's database is represented as a binary array, *databaseStorage*, containing *n* bits of data. Each bit represents one of two types of data:

- Standard data: Represented by 0.
- Priority data: Represented by 1.

To optimize efficiency during peak operational hours, the number of transitions between 0s and 1s in the array must be minimized. Transitions are points in the array where adjacent elements differ (e.g., a 0 followed by a 1 or a 1 followed by a 0).

#### Operations:

- You can swap any two bits in the array as many times as you like.

The goal is to rearrange the bits to minimize transitions with the minimum number of swaps.

#### Example

Consider that *databaseStorage* = [1, 0, 1, 0, 1]. One optimal strategy is to swap *databaseStorage[0]* and *databaseStorage[3]*. After performing this operation, the array is [**0**, 0, 1, **1**, 1], which requires only one transition between *databaseStorage[1]* and *databaseStorage[2]*. This requires only 1 swap, so the return value is 1.

#### Function Description

Complete the function g*etOptimalStorage* in the editor below.

#### Function Parameters

*int databaseStorage[n]:* the current organization of the database's storage, with 0s for standard data and 1s for priority data

### Returns

*int:* the minimum number of operations required to minimize transitions between 0s and 1s

### Constraints

- $2 \le n \le 10^5$
- *databaseStorage[i]* $\in \{0, 1\}$

▼ INPUT FORMAT FOR CUSTOM TESTING

The first line contains an integer, *n,* the size of *databaseStorage*.
Each of the next lines contains an integer, *databaseStorage[i].*

▼ SAMPLE CASE 0

### Sample Input 0

```
STDIN       FUNCTION
-----       --------
7           databaseStorage[] size n = 7
1           databaseStorage = [1, 0, 1, 1, 0, 0, 1]
0
1
1
0
0
1
```

### Sample Output 0

```
1
```

### Explanation

One optimal strategy is to swap *databaseStorage[1]* (which is 0) with *databaseStorage[6]* (which is 1). After performing this operation, the array becomes [1, **1**, 1, 1, 0, 0, **0**]. This solution requires only 1 swap, so the output is 1.

▼ SAMPLE CASE 1

### Sample Input 1

```
STDIN     FUNCTION
-----     --------
9         databaseStorage[] size n = 9
```

```
1
0
0
1
1
0
0
0
1
```

databaseStorage = [1, 0, 0, 1, 1, 0, 0, 0, 1]

### Sample Output 1

```
2
```

### Explanation

One optimal strategy of operations is:

1. Swap *databaseStorage*[1] and *databaseStorage*[8]. After performing this operation, the array becomes [1, **1**, 0, 1, 1, 0, 0, 0, **0**].
2. Swap *databaseStorage*[2] and *databaseStorage*[4]. After performing this operation, the array becomes [1, 1, **1**, 1, **0**, 0, 0, 0, 0].

Now there is only one transition. The minimum number of swaps is 2.

## Interviewer guidelines

▼ SOLUTION

The solution is to count the number of 0s and 1s in the array. Then, we count the number of 0s and 1s that are out of place. The answer is the minimum of the two counts.

```
class Result {

    /*
     * Complete the 'getOptimalStorage' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts INTEGER_ARRAY databaseStorage as parameter.
     */

    public static int getOptimalStorage(List<Integer> databaseStorage) {
        int zeros = 0, ones = 0;
```

```
        for (int brick : databaseStorage) {
            if (brick == 0) {
                zeros++;
            } else {
                ones++;
            }
        }

        int startZero = 0, startOne = 0;

        for (int i = 0; i < ones; ++i) {
            if (databaseStorage.get(i) != 1) {
                startOne++;
            }
        }

        for (int i = 0; i < zeros; ++i) {
            if (databaseStorage.get(i) != 0) {
                startZero++;
            }
        }

        return Math.min(startZero, startOne);
    }
}
```

▼ COMPLEXITY

**Time Complexity -** O(n)

To traverse over the array, we use a loop that runs n times. The complexity is O(n).

**Space Complexity -** O(1)

We use only 2 variables, hence the space complexity is O(1).

**Candidate's Solution**

Language used: **Java 8**

```
import java.io.*;
import java.math.*;
import java.security.*;
import java.text.*;
import java.util.*;
```

```java
import java.util.concurrent.*;
import java.util.function.*;
import java.util.regex.*;
import java.util.stream.*;
import static java.util.stream.Collectors.joining;
import static java.util.stream.Collectors.toList;


class Result {

    /*
     * Complete the 'getOptimalStorage' function below.
     *
     * The function is expected to return an INTEGER.
     * The function accepts INTEGER_ARRAY databaseStorage as parameter.
     */

    public static int getOptimalStorage(List<Integer> databaseStorage) {
        int n=databaseStorage.size();
        int totalOnes=0;
        for(int val:databaseStorage){
            if(val==1)totalOnes++;
        }
        if(totalOnes==0||totalOnes==n)return 0;
        int windowsize=totalOnes;
        int concurrentOnes=0;

        for(int i=0;i<windowsize;i++){
            if(databaseStorage.get(i)==1)concurrentOnes++;

        }
        int  maxOnes=concurrentOnes;
        for(int i=windowsize;i<n;i++){
            if(databaseStorage.get(i)==1)concurrentOnes++;
            if(databaseStorage.get(i-windowsize)==1)concurrentOnes--;
            maxOnes=Math.max(maxOnes,concurrentOnes);
        }
        return totalOnes-maxOnes;

    }

}

public class Solution {
    public static void main(String[] args) throws IOException {
```

```
        BufferedReader bufferedReader = new BufferedReader(new InputStreamReader(System.in));
        BufferedWriter bufferedWriter = new BufferedWriter(new FileWriter(System.getenv("OUTPUT_PATH")));

        int databaseStorageCount = Integer.parseInt(bufferedReader.readLine().trim());

        List<Integer> databaseStorage = IntStream.range(0, databaseStorageCount).mapToObj(i -> {
            try {
                return bufferedReader.readLine().replaceAll("\\s+$", "");
            } catch (IOException ex) {
                throw new RuntimeException(ex);
            }
        })
            .map(String::trim)
            .map(Integer::parseInt)
            .collect(toList());

        int result = Result.getOptimalStorage(databaseStorage);

        bufferedWriter.write(String.valueOf(result));
        bufferedWriter.newLine();

        bufferedReader.close();
        bufferedWriter.close();
    }
}

```

| TESTCASE | DIFFICULTY | TYPE | STATUS | SCORE | TIME TAKEN | MEMORY USED |
|---|---|---|---|---|---|---|
| TestCase 0 | Easy | Sample | Success | 1 | 0.1191 sec | 30.3 KB |
| TestCase 1 | Easy | Sample | Success | 1 | 0.1098 sec | 30.3 KB |

| | | | | | | |
|---|---|---|---|---|---|---|
| TestCase 2 | Easy | Sample | Success | 1 | 0.1218 sec | 30.4 KB |
| TestCase 3 | Easy | Hidden | Success | 3 | 0.1081 sec | 30.5 KB |
| TestCase 4 | Easy | Hidden | Success | 3 | 0.1161 sec | 30 KB |
| TestCase 5 | Easy | Hidden | Success | 3 | 0.094 sec | 30.4 KB |
| TestCase 6 | Easy | Hidden | Success | 4 | 0.1186 sec | 29.8 KB |
| TestCase 7 | Easy | Hidden | Success | 4 | 0.1077 sec | 30 KB |
| TestCase 8 | Easy | Hidden | Success | 5 | 0.1127 sec | 30.3 KB |
| TestCase 9 | Easy | Hidden | Success | 5 | 0.1087 sec | 30.6 KB |
| TestCase 10 | Easy | Hidden | Success | 5 | 0.1034 sec | 30.3 KB |
| TestCase 11 | Medium | Hidden | Wrong Answer | 0 | 0.1431 sec | 31.4 KB |
| TestCase 12 | Medium | Hidden | Wrong Answer | 0 | 0.1316 sec | 31.1 KB |

| | | | | | | |
|---|---|---|---|---|---|---|
| TestCase 13 | Hard | Hidden | Wrong Answer | 0 | 0.1316 sec | 31.3 KB |
| TestCase 14 | Hard | Hidden | Wrong Answer | 0 | 0.313 sec | 54.5 KB |

No comments.